

ASSIGNMENT 1: SQL MASTERY - The E-Commerce Analytics Challenge

Topics Covered

SQL Manipulation, Retrieval, Aggregation, Joins, Subqueries, CTEs, Window Functions, Stored Procedures, Indexes

Duration: 7–8 Hours

Questions: 30

Problem Statement

You're hired as a Data Engineer at "ShopHub" – a fast-growing e-commerce platform. The company has raw transactional data scattered across multiple systems. Your job is to design, optimize, and create a complete analytics database.

Dataset

Brazilian E-Commerce Public Dataset by Olist

Dataset Link:

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Dataset Size: ~100K orders, 8 CSV files (~45 MB)

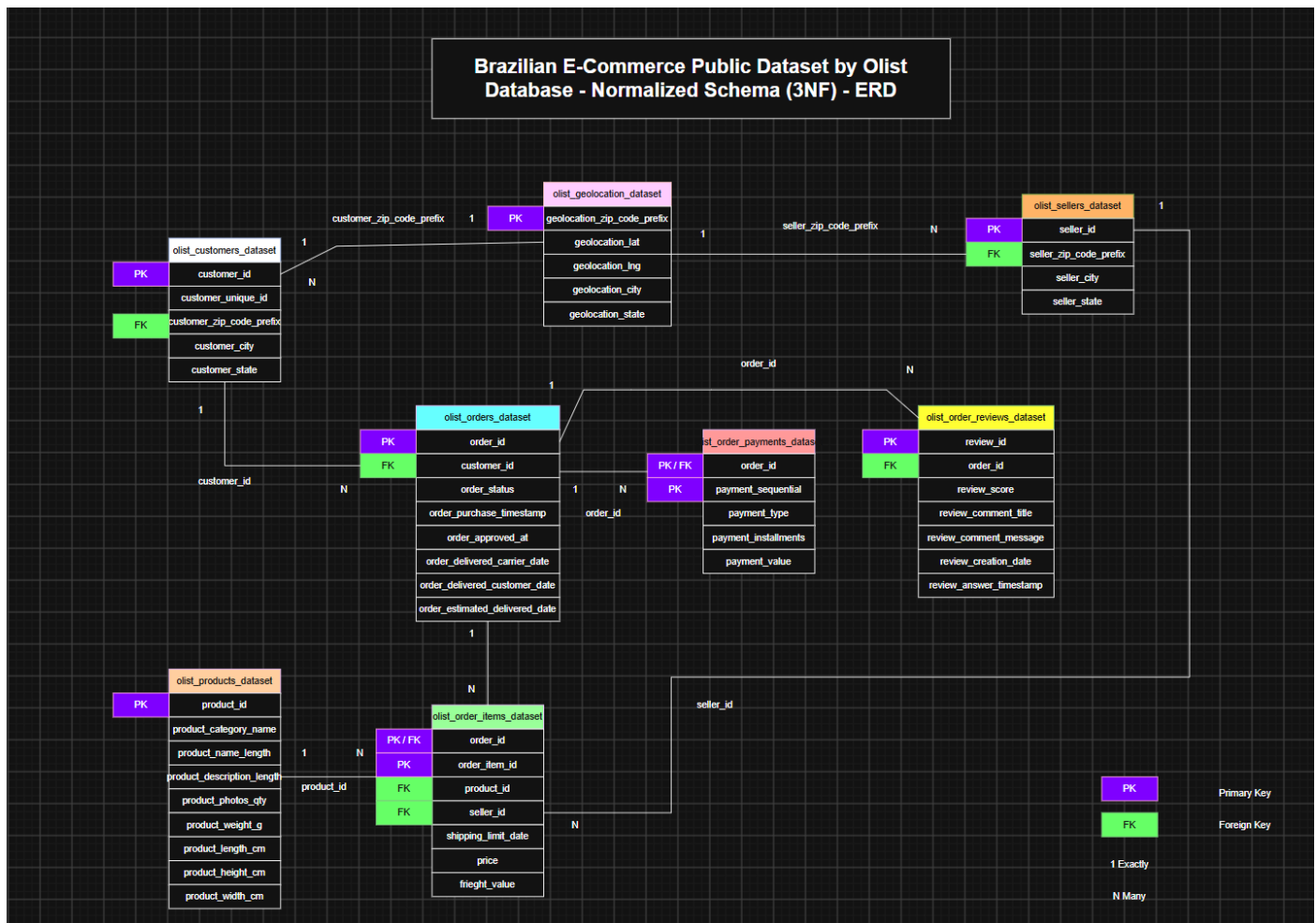
Files Included:

- customers
- orders
- order_items
- products
- sellers
- order_payments
- order_reviews
- geolocation

Assignment Questions

Part A: Database Design & Data Quality (5 Questions)

1. Design a normalized database schema (3NF) with ERD showing all relationships between 8 tables.



2. Identify and document 10+ data quality issues in the raw CSV files.

1) **NULL values in product_category_name**

The product_category_name column in olist_products_dataset contain NULL values.

2) **Incorrect character encoding in city names**

The geolocation_city column in olist_geolocation_dataset contains incorrectly encoded characters. For example, sÃ£o paulo appears instead of sÃ£o paulo.

3) **NULL values in review titles**

The review_comment_title column in olist_order_reviews_dataset contains multiple NULL values.

4) **NULL values in review messages**

The review_comment_message column in olist_order_reviews_dataset contains multiple NULL values.

5) **Potential inconsistency in cancelled orders**

The order_estimated_delivery_date column in olist_orders_dataset is populated for some orders with order_status = cancelled. This should be validated to determine whether the estimated delivery date should be retained for cancelled orders.

6) **NULL values in product photo quantity**

The product_photos_qty column in olist_products_dataset contains NULL values.

7) **NULL values in product name length**

The product_name_length column in olist_products_dataset contains NULL values.

8) **NULL values in product description length**

The product_description_length column in olist_products_dataset contains NULL values.

9) **Invalid/numeric values in seller city**

The seller_city column in olist_sellers_dataset contains numeric/invalid values where city names are expected.

10) **Duplicate records in order items**

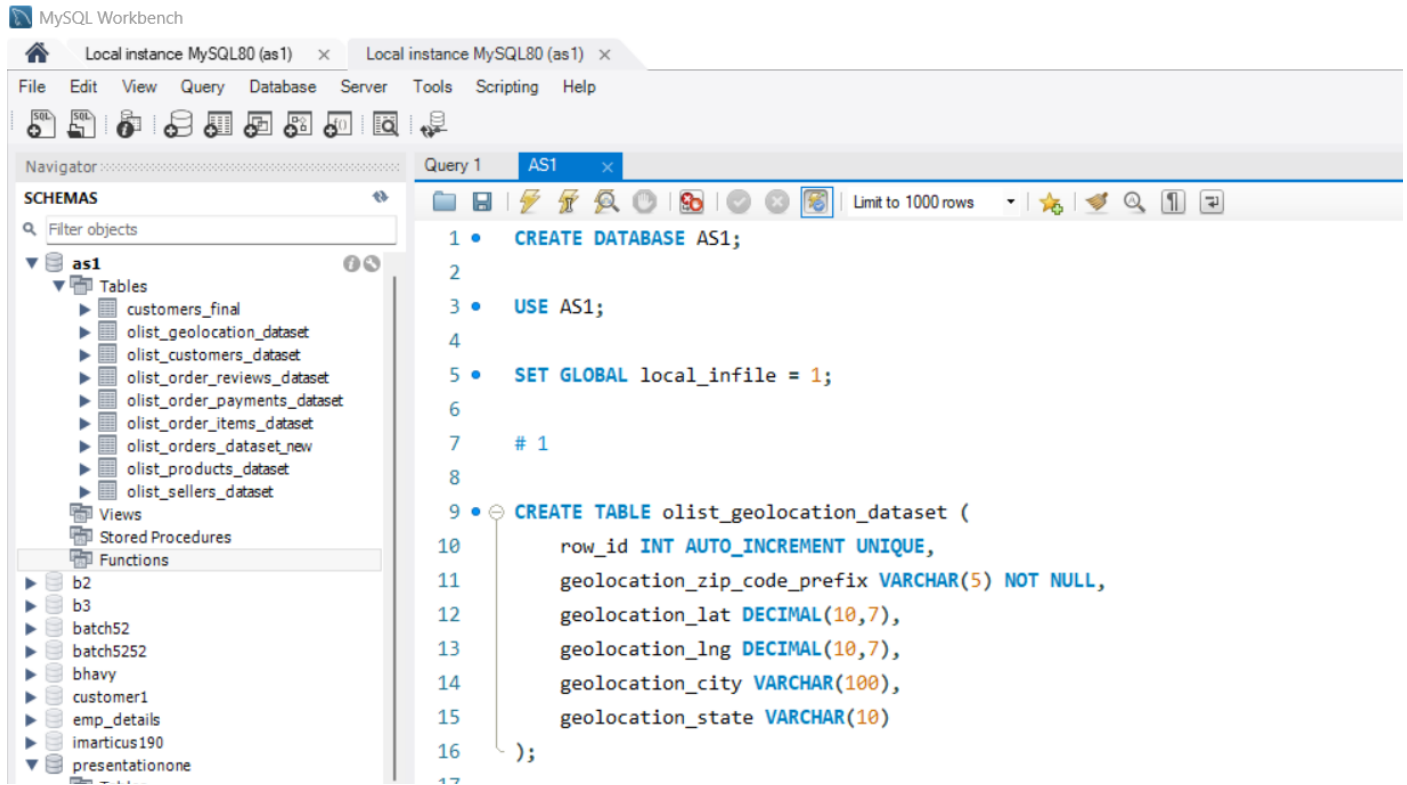
Duplicate rows are present in olist_order_items_dataset, indicating potential duplication in the raw data.

11) **Inconsistent capitalization in city names**

The geolocation_city column in olist_geolocation_dataset contains city names with inconsistent capitalization, such as sao paulo and SAO PAULO. This can cause inconsistencies during grouping, filtering, and analysis.

3. Create tables with appropriate data types, primary keys, foreign keys, and constraints.

Created Database, Use Database, Created Tables, Inserted CSV data using CMD command Prompt



4. Write a SQL script to detect and remove duplicate customers while keeping the most recent record.

```
SELECT
```

```
    customer_unique_id,
```

```
    COUNT(*) AS duplicate_count
```

```
FROM olist_customers_dataset
```

```
GROUP BY customer_unique_id
```

```
HAVING COUNT(*) > 1;
```

```
SELECT *
```

```
FROM olist_customers_dataset
```

```
WHERE customer_unique_id IN (
```

```
    SELECT customer_unique_id
```

```
    FROM olist_customers_dataset
```

```
    GROUP BY customer_unique_id
```

```
    HAVING COUNT(*) > 1
```

```
)
```

```
ORDER BY customer_unique_id, row_id;
```

```
WITH ranked_customers AS (
```

```
    SELECT
```

```
        row_id,
```

```
        customer_unique_id,
```

```
        ROW_NUMBER() OVER (
```

```
            PARTITION BY customer_unique_id
```

```
            ORDER BY row_id DESC
```

```
        ) AS rn
```

```
    FROM olist_customers_dataset
```

```
)
```

```
SELECT *
```

```
FROM ranked_customers
```

```
WHERE rn > 1;
```

```
CREATE TEMPORARY TABLE duplicate_customers AS
```

```
SELECT row_id
```

```
FROM (
```

```
    SELECT
```

```
        row_id,
```

```
        ROW_NUMBER() OVER (
```

```
            PARTITION BY customer_unique_id
```

```
            ORDER BY row_id DESC
```

```
        ) AS rn
```

```
    FROM olist_customers_dataset
```

```
) AS ranked
```

```
WHERE rn > 1;
```

```
DELETE FROM olist_customers_dataset
```

```
WHERE row_id IN (
```

```
    SELECT row_id
```

```
    FROM duplicate_customers
```

```
);
```

```
SELECT
```

```
    customer_unique_id,
```

```
    COUNT(*) AS count
```

```
FROM olist_customers_dataset
```

```
GROUP BY customer_unique_id
```

```
HAVING COUNT(*) > 1;
```

```
SELECT COUNT(*) AS remaining_customers
```

```
FROM olist_customers_dataset;
```

5. Create a comprehensive data dictionary documenting each table, column, data type, and business meaning.

Data Dictionary – Olist E-Commerce Database

1. olist_geolocation_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier used to uniquely identify an imported record.
geolocation_zip_code_prefix	VARCHAR(5)	First five digits of the Brazilian postal code used to identify a geographic area.
geolocation_lat	DECIMAL(10,7)	Latitude coordinate of the geographic location.
geolocation_lng	DECIMAL(10,7)	Longitude coordinate of the geographic location.
geolocation_city	VARCHAR(100)	City associated with the postal code.
geolocation_state	VARCHAR(10)	Brazilian state associated with the location.

2. olist_products_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported product record.
product_id	VARCHAR(32)	Unique identifier of the product.
product_category_name	VARCHAR(100)	Category to which the product belongs.
product_name_length	INT	Number of characters in the product name.
product_description_length	INT	Number of characters in the product description.
product_photos_qty	INT	Number of photographs available for the product.
product_weight_g	INT	Product weight in grams.
product_length_cm	INT	Product length in centimeters.
product_height_cm	INT	Product height in centimeters.
product_width_cm	INT	Product width in centimeters.

3. olist_customers_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported customer record.
customer_id	VARCHAR(32)	Identifier assigned to a customer for an order.

Column	Data Type	Business Meaning
customer_unique_id	VARCHAR(32)	Unique identifier representing the customer across orders.
customer_zip_code_prefix	VARCHAR(5)	First five digits of the customer's postal code.
customer_city	VARCHAR(100)	City where the customer is located.
customer_state	VARCHAR(10)	Brazilian state where the customer is located.

Important: customer_id identifies a customer record associated with an order, while customer_unique_id is used to identify the actual customer across multiple orders.

4. olist_sellers_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported seller record.
seller_id	VARCHAR(32)	Unique identifier of the seller.
seller_zip_code_prefix	VARCHAR(5)	First five digits of the seller's postal code.
seller_city	VARCHAR(100)	City where the seller is located.
seller_state	VARCHAR(10)	Brazilian state where the seller is located.

5. olist_orders_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported order record.
order_id	VARCHAR(32)	Unique identifier of an order.
customer_id	VARCHAR(32)	Identifier of the customer who placed the order.
order_status	VARCHAR(30)	Current status of the order, such as delivered, shipped, canceled, etc.
order_purchase_timestamp	DATETIME	Date and time when the customer placed the order.
order_approved_at	DATETIME	Date and time when the payment/order was approved.
order_delivered_carrier_date	DATETIME	Date and time when the order was handed over to the carrier.
order_delivered_customer_date	DATETIME	Date and time when the order was delivered to the customer.
order_estimated_delivery_date	DATETIME	Estimated date on which the order was expected to be delivered.

6. olist_order_items_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported order-item record.

Column	Data Type	Business Meaning
order_id	VARCHAR(32)	Identifier of the order containing the item.
order_item_id	INT	Sequential number identifying an item within an order.
product_id	VARCHAR(32)	Identifier of the product purchased.
seller_id	VARCHAR(32)	Identifier of the seller who sold the product.
shipping_limit_date	DATETIME	Deadline by which the seller should ship the item.
price	DECIMAL(10,2)	Price of the purchased product.
freight_value	DECIMAL(10,2)	Shipping/freight cost charged for the item.

7. olist_order_payments_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported payment record.
order_id	VARCHAR(32)	Identifier of the order associated with the payment.
payment_sequential	INT	Sequential number of a payment within an order.
payment_type	VARCHAR(30)	Payment method used, such as credit card, boleto, voucher, or debit card.
payment_installments	INT	Number of installments used for the payment.
payment_value	DECIMAL(10,2)	Monetary value of the payment.

8. olist_order_reviews_dataset

Column	Data Type	Business Meaning
row_id	INT	Auto-generated row identifier for the imported review record.
review_id	VARCHAR(32)	Identifier of the customer review.
order_id	VARCHAR(32)	Identifier of the order being reviewed.
review_score	INT	Customer rating given to the order, typically from 1 to 5.
review_comment_title	VARCHAR(255)	Short title/headline provided with the review.
review_comment_message	TEXT	Detailed written feedback provided by the customer.
review_creation_date	DATETIME	Date and time when the review was created.
review_answer_timestamp	DATETIME	Date and time when the review was answered.

Part B: Data Manipulation & Retrieval (5 Questions)

6. Insert cleaned data using transactions and demonstrate COMMIT and ROLLBACK.

```
SET autocommit = 0;
```

```
START TRANSACTION;
```

```
CREATE TABLE customers_final (  
    customer_id VARCHAR(32) PRIMARY KEY,  
    customer_unique_id VARCHAR(32) NOT NULL,  
    customer_zip_code_prefix VARCHAR(5),  
    customer_city VARCHAR(100),  
    customer_state VARCHAR(10)  
);
```

```
INSERT INTO customers_final (  
    customer_id,  
    customer_unique_id,  
    customer_zip_code_prefix,  
    customer_city,  
    customer_state  
)
```

```
SELECT  
    customer_id,  
    customer_unique_id,  
    customer_zip_code_prefix,  
    customer_city,  
    customer_state  
FROM olist_customers_dataset;
```

```
SELECT COUNT(*) FROM customers_final;
```

```
ROLLBACK;
```

```
SELECT COUNT(*) FROM customers_final;
```

```
START TRANSACTION;
```

```
INSERT INTO customers_final (
```

```
    customer_id,
```

```
    customer_unique_id,
```

```
    customer_zip_code_prefix,
```

```
    customer_city,
```

```
    customer_state
```

```
)
```

```
SELECT
```

```
    customer_id,
```

```
    customer_unique_id,
```

```
    customer_zip_code_prefix,
```

```
    customer_city,
```

```
    customer_state
```

```
FROM olist_customers_dataset;
```

```
SELECT COUNT(*) FROM customers_final;
```

```
COMMIT;
```

```
SELECT COUNT(*) FROM customers_final;
```

```
SET autocommit = 1;
```

7. Find customers who have the same email but different names/addresses.

NO VALID COLUMNS

8. Identify orphan records referencing non-existent products or orders.

no orphan records were found.

9. List customers who registered but never placed an order.

No customers were found who registered but never placed an order.

10. Detect potential fraud where payment value differs from order total.

303 orders were identified where the total payment value differs from the calculated order total (price + freight_value) by more than 0.01.

WITH order_totals AS (

SELECT

order_id,

SUM(price + freight_value) AS order_total

FROM olist_order_items_dataset

GROUP BY order_id

),

payment_totals AS (

SELECT

order_id,

SUM(payment_value) AS payment_total

FROM olist_order_payments_dataset

GROUP BY order_id

)

SELECT

o.order_id,

ROUND(o.order_total, 2) AS order_total,

ROUND(p.payment_total, 2) AS payment_total,

ROUND(p.payment_total - o.order_total, 2) AS difference

FROM order_totals AS o

JOIN payment_totals AS p

ON o.order_id = p.order_id

WHERE ABS(p.payment_total - o.order_total) > 0.01

ORDER BY ABS(p.payment_total - o.order_total) DESC;

Result Grid	Filter Rows:	Exports:	Wrap Cell Contents:
order_id	order_total	payment_total	difference
ce6d150fb29ada17d2082f4847107665	1403.66	1586.47	182.81
6e5fe7366a2e1bfbf3257dba0af1267f	287.91	406.92	119.01
70b742795bc441e94a44a084b6d9ce7a	466.93	578.82	111.89
996c7e73600ad3723e8627ab7bef81e4	587.90	664.43	76.53
70b7e94ea46d3e8b5bc12a50186edaf0	213.15	274.84	61.69
bc2c82b0ef78d2252b6176d1972db7c9	242.01	303.02	61.01
af9ffff2ce6b3defd34fd4c78857a379	413.17	466.97	53.80
262118ce178bb3e4590a3adc6d62e6b	177.74	126.12	-51.62
bfd55bb06458d600a33d61f5f287472	348.93	394.36	45.43
8d9c0dc8d5a2ce804f6b925d8f8e6c1d	254.45	293.89	39.44
b7579d24f5b2dd3e20f2e57d0e07d170	466.28	504.37	38.09
abf1130bc676c9dcadf91e24f5e30a30	459.36	496.89	37.53
b76190a2c095fba255ab46987545a660	227.91	263.24	35.33
32720c0c8b42f4c2c0b07cee5ec2444b	224.75	259.59	34.84
6h0ac1hrdea327c7060eaa92feh9e9a6f	327.87	367.56	34.69

Part C: Complex Aggregations (5 Questions)

11. Calculate monthly revenue with Month-over-Month growth percentage.

WITH monthly_revenue AS (

SELECT

DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS month,

SUM(oi.price + oi.freight_value) AS revenue

FROM olist_orders_dataset_new AS o

JOIN olist_order_items_dataset AS oi

ON o.order_id = oi.order_id

GROUP BY DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')

),

monthly_growth AS (

SELECT

month,

revenue,

LAG(revenue) OVER (ORDER BY month) AS previous_month_revenue

FROM monthly_revenue

)

SELECT

month,

ROUND(revenue, 2) AS revenue,

ROUND(previous_month_revenue, 2) AS previous_month_revenue,

ROUND(

((revenue - previous_month_revenue)

/ previous_month_revenue) * 100,

2

) AS mom_growth_percentage

FROM monthly_growth

ORDER BY month;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
month	revenue	previous_month_revenue	mom_growth_percentage
2016-09	354.75	NULL	NULL
2016-10	56808.84	354.75	15913.77
2016-12	19.62	56808.84	-99.97
2017-01	137188.49	19.62	699127.78
2017-02	286280.62	137188.49	108.68
2017-03	432048.59	286280.62	50.92
2017-04	412422.24	432048.59	-4.54
2017-05	586190.95	412422.24	42.13
2017-06	502963.04	586190.95	-14.20

12. Find top 10 products by revenue in each category.

```
WITH product_revenue AS (  
    SELECT  
        p.product_category_name,  
        oi.product_id,  
        SUM(oi.price) AS revenue  
    FROM olist_order_items_dataset AS oi  
    JOIN olist_products_dataset AS p  
        ON oi.product_id = p.product_id  
    GROUP BY  
        p.product_category_name,  
        oi.product_id  
)  
ranked_products AS (  
    SELECT  
        product_category_name,  
        product_id,  
        revenue,  
        ROW_NUMBER() OVER (  
            PARTITION BY product_category_name  
            ORDER BY revenue DESC  
        ) AS product_rank  
    FROM product_revenue  
)  
SELECT  
    product_category_name,  
    product_id,  
    ROUND(revenue, 2) AS revenue,  
    product_rank  
FROM ranked_products  
WHERE product_rank <= 10  
ORDER BY  
    product_category_name,  
    product_rank;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	product_category_name	product_id	revenue	product_rank
▶		5a848e4ab52fd5445cdc07aab1c40e48	24229.03	1
		eed5cbd74fac3bd79b7c7ec95fa7507d	9945.00	2
		b1d207586fca400a2370d50a9ba1da98	7152.00	3
		76d1a1a9d21ab677a61c3ae34b1b352f	5712.64	4
		ad88641611c35ebd59ecda07a9f17099	4515.33	5
		3b60d513e90300a4e9833e5cda1f1d61	4393.33	6
		4c50dcc50f1512f46096d6ef0142c4a9	3980.00	7
		17823ffd2de8234f0e885a71109613a4	2969.89	8
		0e030462875259ec0cb868f7ecf1fd5e	2740.00	9
		b36f3c918c91478c4559160022d3f14e	2550.00	10
	agro_industria_e_come...	11250b0d4b709fee92441c5f34122aed	9111.00	1
	agro_industria_e_come...	423a6644f0aa529e8828ff1f91003690	8043.00	2
	agro_industria_e_come...	672e757f331900b9deea127a2a7b79fd	6885.00	3
	agro_industria_e_come...	c183fd5d2abf05873fa6e1014ed9e06c	5934.60	4
	agro_industria_e_come...	2h69866f22de8dad69c976771daha91c	2990.00	5

Result 62 

13. Calculate Customer Lifetime Value (CLV) and categorize customers.

Step 1 — Calculate each customer's total spending

WITH customer_spending AS (

SELECT

o.customer_id,

SUM(oi.price + oi.freight_value) AS total_spending

FROM olist_orders_dataset AS o

JOIN olist_order_items_dataset AS oi

ON o.order_id = oi.order_id

GROUP BY o.customer_id

)

SELECT

customer_id,

ROUND(total_spending, 2) AS CLV

FROM customer_spending

ORDER BY CLV DESC;

Result Grid Filter Rows: Export: Wrap Cell Content:				
	product_category_name	product_id	revenue	product_rank
▶		5a848e4ab52fd5445cdc07aab1c40e48	24229.03	1
		eed5cbd74fac3bd79b7c7ec95fa7507d	9945.00	2
		b1d207586fca400a2370d50a9ba1da98	7152.00	3
		76d1a1a9d21ab677a61c3ae34b1b352f	5712.64	4
		ad88641611c35ebd59ecda07a9f17099	4515.33	5
		3b60d513e90300a4e9833e5cda1f1d61	4393.33	6
		4c50dcc50f1512f46096d6ef0142c4a9	3980.00	7
		17823ffd2de8234f0e885a71109613a4	2969.89	8
		0e030462875259ec0cb868f7ecf1fd5e	2740.00	9
		b36f3c918c91478c4559160022d3f14e	2550.00	10
	agro_industria_e_come...	11250b0d4b709fee92441c5f34122aed	9111.00	1
	agro_industria_e_come...	423a6644f0aa529e8828ff1f91003690	8043.00	2
	agro_industria_e_come...	672e757f331900b9deea127a2a7b79fd	6885.00	3
	agro_industria_e_come...	c183fd5d2abf05873fa6e1014ed9e06c	5934.60	4
	agro_industria_e_come...	2b69866f22de8dad69c976771daha91c	2990.00	5

Result 61 x

Step 2 — Categorize customers

WITH customer_spending AS (

```

SELECT
    o.customer_id,
    SUM(oi.price + oi.freight_value) AS CLV
FROM olist_orders_dataset AS o
JOIN olist_order_items_dataset AS oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id
)

```

```

SELECT
    customer_id,
    ROUND(CLV, 2) AS CLV,
CASE
    WHEN CLV >= 1000 THEN 'High Value'
    WHEN CLV >= 500 THEN 'Medium Value'
    ELSE 'Low Value'
END AS customer_category
FROM customer_spending
ORDER BY CLV DESC;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	customer_id	CLV	customer_category
▶	1617b1357756262bfa56ab541c47bc16	13664.08	High Value
	ec5b2ba62e574342386871631fafd3fc	7274.88	High Value
	c6e2731c5b391845f6800c97401a43a9	6929.31	High Value
	f48d464a0baaea338cb25f816991ab1f	6922.21	High Value
	3fd6777bbce08a352fdd04e4a7cc8f6	6726.66	High Value
	05455dfa7cd02f13d132aa7a6a9729c6	6081.54	High Value
	df55c14d1476a9a3467f131269c2477f	4950.34	High Value
	e0a2412720e9ea4f26c1ac985f6a7358	4809.44	High Value
	24bbf5fd2f2e1b359ee7de94defc4a15	4764.34	High Value
	3d979689f636322c62418b6346b1c6d2	4681.78	High Value
	1afc82cd60e303ef09b4ef9837c9505c	4513.32	High Value
	cc803a2c412833101651d3f90ca7de24	4445.50	High Value
	926b6a6fb8b6081e00b335edaf578d35	4194.76	High Value
	35a413c7ca3c69756cb75867d6311c0d	4175.26	High Value
	e9h0d0eh3015ef1c9ce6cf5h9dcbee9f	4163.51	Hinh Value

Result 60

×

14. Create sales reports using ROLLUP or CUBE.

SELECT

```
YEAR(o.order_purchase_timestamp) AS year,  
MONTH(o.order_purchase_timestamp) AS month,  
ROUND(SUM(oi.price + oi.freight_value), 2) AS total_sales
```

FROM olist_orders_dataset_new AS o

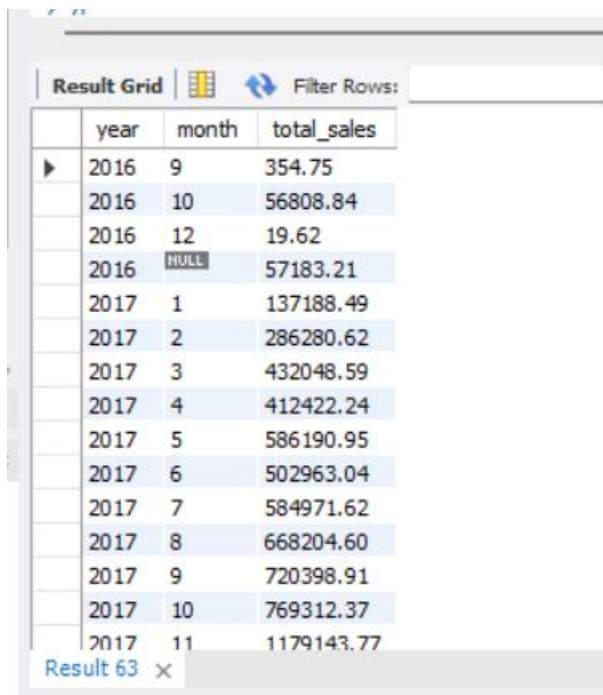
JOIN olist_order_items_dataset AS oi

```
ON o.order_id = oi.order_id
```

GROUP BY

```
YEAR(o.order_purchase_timestamp),  
MONTH(o.order_purchase_timestamp)
```

WITH ROLLUP;



The screenshot shows a 'Result Grid' window with a table of sales data. The table has three columns: 'year', 'month', and 'total_sales'. The data is grouped by year and month, with a ROLLUP operation applied to the year 2016, resulting in a 'NULL' month entry for that year. The total sales for 2016 are 57183.21. The data for 2017 shows months 1 through 11, with total sales values ranging from 137188.49 to 1179143.77. The window title is 'Result 63'.

year	month	total_sales
2016	9	354.75
2016	10	56808.84
2016	12	19.62
2016	NULL	57183.21
2017	1	137188.49
2017	2	286280.62
2017	3	432048.59
2017	4	412422.24
2017	5	586190.95
2017	6	502963.04
2017	7	584971.62
2017	8	668204.60
2017	9	720398.91
2017	10	769312.37
2017	11	1179143.77

15. Identify seasonal sales patterns by product category.

SELECT

MONTH(o.order_purchase_timestamp) AS month,

p.product_category_name,

ROUND(SUM(oi.price + oi.freight_value), 2) AS total_sales

FROM olist_orders_dataset_new AS o

JOIN olist_order_items_dataset AS oi

ON o.order_id = oi.order_id

JOIN olist_products_dataset AS p

ON oi.product_id = p.product_id

GROUP BY

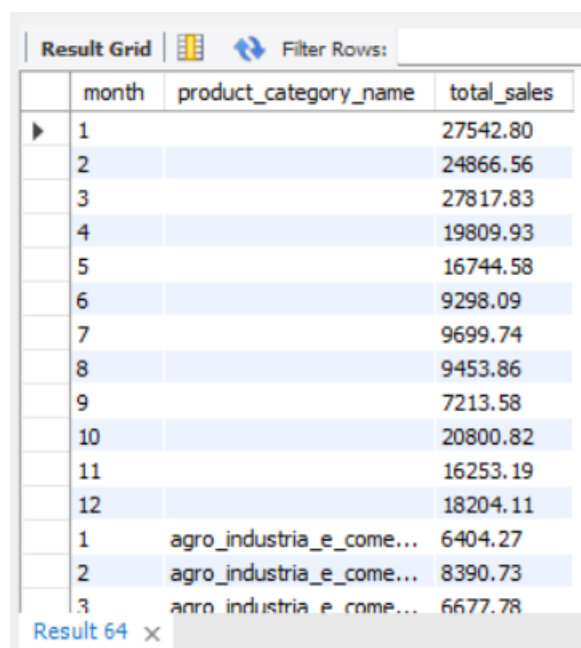
MONTH(o.order_purchase_timestamp),

p.product_category_name

ORDER BY

p.product_category_name,

month;



The screenshot shows a SQL query result grid with the following columns: month, product_category_name, and total_sales. The data is grouped by month and product category. The first 12 rows show monthly totals for months 1 through 12. The next 3 rows show monthly totals for the product category 'agro_industria_e_come...' for months 1, 2, and 3.

	month	product_category_name	total_sales
▶	1		27542.80
	2		24866.56
	3		27817.83
	4		19809.93
	5		16744.58
	6		9298.09
	7		9699.74
	8		9453.86
	9		7213.58
	10		20800.82
	11		16253.19
	12		18204.11
	1	agro_industria_e_come...	6404.27
	2	agro_industria_e_come...	8390.73
	3	agro_industria_e_come...	6677.78

Result 64 x

Part D: Mastering Joins (5 Questions)

16. Create a customer 360-degree view joining multiple tables.

SELECT

```
c.customer_id,  
c.customer_unique_id,  
c.customer_city,  
c.customer_state,  
  
o.order_id,  
o.order_status,  
o.order_purchase_timestamp,  
  
oi.product_id,  
p.product_category_name,  
oi.price,  
oi.freight_value,  
  
pay.payment_type,  
pay.payment_value,  
  
r.review_score
```

FROM olist_customers_dataset AS c

LEFT JOIN olist_orders_dataset AS o

ON c.customer_id = o.customer_id

LEFT JOIN olist_order_items_dataset AS oi

ON o.order_id = oi.order_id

LEFT JOIN olist_products_dataset AS p

ON oi.product_id = p.product_id

LEFT JOIN olist_order_payments_dataset AS pay

ON o.order_id = pay.order_id

LEFT JOIN olist_order_reviews_dataset AS r

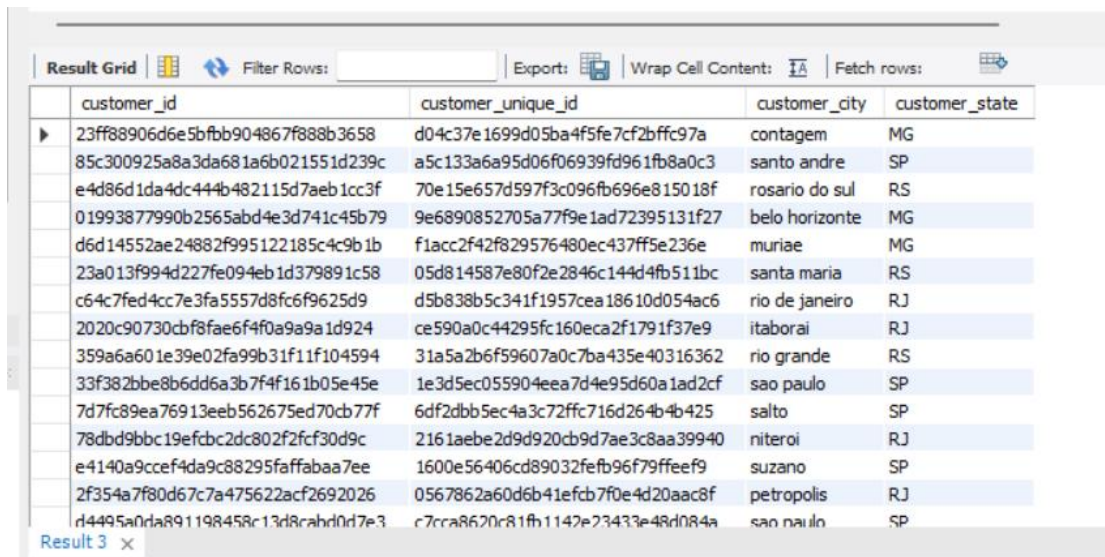
ON o.order_id = r.order_id

ORDER BY c.customer_id, o.order_purchase_timestamp;

customer_id	customer_unique_id	customer_city	customer_state	order_id	order_status	order_purchase_timestamp
00012a2ce6f8dcda20d059ce98491703	248ffe10d632bebe4f7267f1f44844c9	osasco	SP	5f79b5b0931d63f1a42989eb65b9da6e	delivered	2017-11-14 16:08:00
000161a058600d5901f007fab4c27140	b0015e09bb4b6e47c52844fab5fb6638	itapecerica	MG	a44895d095d7e0702b6a162fa2dbeced	delivered	2017-07-16 09:40:00
0001fd6190edaa884bcdf3d49edf079	94b11d37cd61cb2994a194d11f89682b	nova venecia	ES	316a104623542e4d75189bb372bc5f8d	delivered	2017-02-28 11:06:00
0002414f95344307404f0ace7a26f1d5	4893ad4ea28b2c5b3ddf4e82e79db9e6	mendonca	MG	5825ce2e88d5346438686b0bba99e5ee	delivered	2017-08-16 13:09:00
000379cdec625522490c315e70c7a9fb	0b83f73b19c2019e182fd552c048a22c	sao paulo	SP	0ab7fb08086d4af9141453c91878ed7a	delivered	2018-04-02 13:42:00
0004164d20a9e969af783496f3408652	104bdb7e6a6cdceaa88c3ea5fa6b2b93	valinhos	SP	cd3558a10d854487b4f907e9b326a4fc	delivered	2017-04-12 08:35:00
00046a560d407e99b969756e0b10f282	0b5295fc9819d831f68eb0e9a3e13ab7	rio de janeiro	RJ	8c3d752c5c02227878fae49aeaddbdf7	delivered	2017-12-18 11:08:00
00050bf6e01e69d5c0fd612f1bcfb69c	e3cf594a99e810f58af53ed4820f25e5	ijui	RS	fa906f338cee30a984d0945b3832e431	delivered	2017-09-17 16:04:00
000598caf2ef4117407665ac33275130	7e0516b486e92ed3f3afdd6d1276cfbd	oliveira	MG	9b961b894e797f63622137ff7eb1c1af	delivered	2018-08-11 12:14:00
0005aefbb696d34b3424ccdd0a0e9fd0	616309b2eeb7bd9c05b0fdfbab28e6c6	sao paulo	SP	263ba12390d0fbce329dd16da8cd20f8	delivered	2018-06-20 09:46:00
00062b33cb9f6fe976afdcff967ea74d	f90f55ee274a4ae21510b386134b09cd	sao paulo	SP	c208db5638f7f1cd04d185856852f864	delivered	2017-03-15 23:44:00
00066ccbe787a588c52bd5ff404590e3	15090f48004f3b0fc18a167ef82af4db	novo hamburgo	RS	728416b0db65935dbf78a0cc03e8d6f8	delivered	2018-02-06 16:10:00
00066ccbe787a588c52bd5ff404590e3	15090f48004f3b0fc18a167ef82af4db	novo hamburgo	RS	728416b0db65935dbf78a0cc03e8d6f8	delivered	2018-02-06 16:10:00
00066c787a588c52bd5ff404590e3	15090f48004f3b0fc18a167ef82af4db	novo hamburgo	RS	728416b0db65935dbf78a0cc03e8d6f8	delivered	2018-02-06 16:10:00

17. Find customers who bought electronics but never books.

```
SELECT DISTINCT
    c.customer_id,
    c.customer_unique_id,
    c.customer_city,
    c.customer_state
FROM olist_customers_dataset AS c
JOIN olist_orders_dataset_new AS o
    ON c.customer_id = o.customer_id
JOIN olist_order_items_dataset AS oi
    ON o.order_id = oi.order_id
JOIN olist_products_dataset AS p
    ON oi.product_id = p.product_id
WHERE p.product_category_name LIKE '%eletron%'
AND c.customer_id NOT IN (
    SELECT DISTINCT o2.customer_id
    FROM olist_orders_dataset_new AS o2
    JOIN olist_order_items_dataset AS oi2
        ON o2.order_id = oi2.order_id
    JOIN olist_products_dataset AS p2
        ON oi2.product_id = p2.product_id
    WHERE p2.product_category_name LIKE '%livro%'
);
```



	customer_id	customer_unique_id	customer_city	customer_state
▶	23ff88906d6e5bfb904867f888b3658	d04c37e1699d05ba4f5fe7cf2bffc97a	contagem	MG
	85c300925a8a3da681a6b021551d239c	a5c133a6a95d06f06939fd961fb8a0c3	santo andre	SP
	e4d86d1da4dc444b482115d7aeb1cc3f	70e15e657d597f3c096fb696e815018f	rosario do sul	RS
	01993877990b2565abd4e3d741c45b79	9e6890852705a77f9e1ad72395131f27	belo horizonte	MG
	d6d14552ae24882f995122185c4c9b1b	f1acc2f42f829576480ec437ff5e236e	muriae	MG
	23a013f994d227fe094eb1d379891c58	05d814587e80f2e2846c144d4fb511bc	santa maria	RS
	c64c7fed4cc7e3fa5557d8fc6f9625d9	d5b838b5c341f1957cea18610d054ac6	rio de janeiro	RJ
	2020c90730cbf8fae6f4f0a9a1d924	ce590a0c44295fc160eca2f1791f37e9	itaborai	RJ
	359a6a601e39e02fa99b31f11f104594	31a5a2b6f59607a0c7ba435e40316362	rio grande	RS
	33f382bbe8b6dd6a3b7f4f161b05e45e	1e3d5ec055904eea7d4e95d60a1ad2cf	sao paulo	SP
	7d7fc89ea76913eeb562675ed70cb77f	6df2dbb5ec4a3c72ffc716d264b4b425	salto	SP
	78dbd9bbc19efcb2dc802f2fc30d9c	2161aeb2d9d920cb9d7ae3c8aa39940	niteroi	RJ
	e4140a9cce4da9c88295faffabaa7ee	1600e56406cd89032febf96f79ffef9	suzano	SP
	2f354a7f80d67c7a475622acf2692026	0567862a60d6b41efcb7f0e4d20aac8f	petropolis	RJ
	d4495a0da891198458c13d8c9ad0d7e3	c7cca8620c81fb1142e23433e48d084a	sao paulo	SP

18. List sellers and their best-selling product in each category.

```
WITH product_sales AS (  
    SELECT  
        oi.seller_id,  
        p.product_category_name,  
        oi.product_id,  
        COUNT(*) AS units_sold  
    FROM olist_order_items_dataset AS oi  
    JOIN olist_products_dataset AS p  
        ON oi.product_id = p.product_id  
    GROUP BY  
        oi.seller_id,  
        p.product_category_name,  
        oi.product_id  
)  
ranked_products AS (  
    SELECT  
        seller_id,  
        product_category_name,  
        product_id,  
        units_sold,  
        ROW_NUMBER() OVER (  
            PARTITION BY seller_id, product_category_name  
            ORDER BY units_sold DESC  
        ) AS product_rank  
    FROM product_sales  
)  
SELECT  
    seller_id,  
    product_category_name,  
    product_id,  
    units_sold  
FROM ranked_products
```

WHERE product_rank = 1

ORDER BY

seller_id,

product_category_name,

units_sold DESC;

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	Fetch rows:
seller_id	product_category_name	product_id	units_sold	
0015a82c2db000af6aaaf3ae2ecb0532	eletroportateis	a2ff5a97bf95719e38ea2e3b4105bce8	3	
001cca7ae9ae17fb1caed9dfb1094831	construcao_ferramentas_construcao	0da9ffd92214425d880de3f94e74ce39	17	
001cca7ae9ae17fb1caed9dfb1094831	ferramentas_jardim	08574b074924071f4e201e151b152b4e	113	
001e6ad469a905060d959994f1b41e4f	esporte_lazer	093cd981b714bcdff182b427d87fc8fc	1	
002100f778ceb8431b7a1020ff7ab48f	moveis_decoracao	158102fe543dbaeb84d87811bfe06d0d	17	
003554e2dce176b5555353e4f3555ac8		67f36b3689147d882d2b298fd0715d80	1	
004c9cd9d87a3c30c522c48c4fc07416		4257f45f6ea58aa46885ed7ef723b6c0	4	
004c9cd9d87a3c30c522c48c4fc07416	cama_mesa_banho	54af11c815f92135fb554bd9c97dd418	13	
00720abe85ba0859807595bbf045a33b	utilidades_domesticas	0e14941c183946f60e3b86585d05f984	5	
00ab3eff1b5192e5f1a63bcecfef11c8	beleza_saude	362db8a45afd47d3f54c0f34f7c828bc	1	
00d8b143d12632bad99c0ad66ad52825	fashion_bolsas_e_acessorios	e4db070498b75e666a0736eb2f0abcd4	1	
00ee68308b45bc5e2660cd833c3f81cc	cama_mesa_banho	d692d19cee022f7acab76373f297ad6a	22	
00ee68308b45bc5e2660cd833c3f81cc	moveis_colchao_e_estofado	726b4e18f00255e2e63491bcba3f60b8	22	
00fc707aaaad2d31347cf883cd2dfe10	informatica_acessorios	656e0eca68dcecf6a31b8ececfa3e8	25	
010543a62hd80aa422851e79a3hc7540	informatica_acessorios	cd8ch942h6a58frff9697762d3hc9cd7	2	

Result 7

19. Perform market basket analysis using self-joins.

```
SELECT
    oi1.product_id AS product_1,
    oi2.product_id AS product_2,
    COUNT(DISTINCT oi1.order_id) AS times_bought_together
FROM olist_order_items_dataset AS oi1
JOIN olist_order_items_dataset AS oi2
    ON oi1.order_id = oi2.order_id
    AND oi1.product_id < oi2.product_id
GROUP BY
    oi1.product_id,
    oi2.product_id
ORDER BY
    times_bought_together DESC;
```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	product_1	product_2	times_bought_together				
▶	36f60d45225e60c7da4558b070ce4b60	e53e557d5a159f5aa2c5e995dfdf244b	34				
	35afc973633aaeb6b877ff57b2793310	99a4788cb24856965c36a24e339b6058	29				
	4fcb3d9a5f4871e8362dfedbdb02b064	f4f67ccaece962d013a4e1d7dc3a61f7	17				
	36f60d45225e60c7da4558b070ce4b60	3f14d740544f37ece8a9e7bc8349797e	12				
	389d119b48cf3043d311335e499d9c6b	422879e10f46682990de24d770e7f83d	11				
	389d119b48cf3043d311335e499d9c6b	53759a2ecddad2bb87a079a1f1519f73	9				
	368c6c730842d78016ad823897a372db	53759a2ecddad2bb87a079a1f1519f73	8				
	18486698933fbb64af6c0a255f7dd64c	dbb67791e405873b259e4656bf971246	7				
	422879e10f46682990de24d770e7f83d	53759a2ecddad2bb87a079a1f1519f73	7				
	060cb19345d90064d1015407193c233d	98d61056e0568ba048e5d78038790e77	6				
	0bcc3eeca39e1064258aa1e932269894	422879e10f46682990de24d770e7f83d	6				
	0d85c435fd60b277ffb9e9b0f88f927a	ee57070aa3b24a06fdd0e02efd2d757d	6				
	4d0ec1e9b95fb62f9a1fbc21808bf3b1	9ad75bd7267e5c724cb42c71ac56ca72	6				
	5b8a5a9417210b1b84b67b9a7aefb935	e5ae72c62ebfa708624f5029d609b160	6				
	5d790355cheded0cd60e25c4c4c527a2	5fc3e6a4b52b0c414458104ed4037f1c	6				

Result 8 x

20. Identify orders with shipping delays.

SELECT

```
order_id,  
order_status,  
order_delivered_customer_date,  
order_estimated_delivery_date,  
DATEDIFF(  
    STR_TO_DATE(order_delivered_customer_date, '%d-%m-%Y %H:%i'),  
    STR_TO_DATE(order_estimated_delivery_date, '%d-%m-%Y %H:%i')  
) AS delay_days
```

FROM olist_orders_dataset_new

WHERE STR_TO_DATE(order_delivered_customer_date, '%d-%m-%Y %H:%i')

> STR_TO_DATE(order_estimated_delivery_date, '%d-%m-%Y %H:%i')

ORDER BY delay_days DESC;

order_id	order_status	order_delivered_customer_date	order_estimated_delivery_date	delay_days
1b3190b2dfa9d789e1f14c05b647a14a	delivered	19-09-2018 23:24	15-03-2018 00:00	188
ca07593549f1816d26a572e06dc1eab6	delivered	19-09-2017 14:36	22-03-2017 00:00	181
47b40429ed8cce3aee9199792275433f	delivered	13-07-2018 20:51	19-01-2018 00:00	175
2fe324feb907e3ea3f2aa9650869fa5	delivered	19-09-2017 17:00	05-04-2017 00:00	167
285ab9426d6982034523a855f55a885e	delivered	19-09-2017 14:00	06-04-2017 00:00	166
440d0d17af552815d15a9e41abe49359	delivered	19-09-2017 15:12	07-04-2017 00:00	165
c27815f7e3dd0b926b58552628481575	delivered	19-09-2017 17:14	10-04-2017 00:00	162
0f4519c5f1c541ddec9f21b3bddd533a	delivered	19-09-2017 14:38	11-04-2017 00:00	161
d24e8541128cea179a11a65176e0a96f	delivered	04-12-2017 18:36	26-06-2017 00:00	161
2d7561026d542c8dbd8f0daeadf67a43	delivered	19-09-2017 14:38	13-04-2017 00:00	159
2fb597c2f772eca01b1f5c561bf6cc7b	delivered	19-09-2017 14:33	17-04-2017 00:00	155
6e82dcfb5eada6283dba34f164e636f5	delivered	16-11-2017 10:56	14-06-2017 00:00	155

Part E: Subqueries & CTEs (4 Questions)

21. Find customers spending above their state's average.

```
WITH customer_spending AS (  
    SELECT  
        c.customer_id,  
        c.customer_state,  
        SUM(oi.price + oi.freight_value) AS total_spending  
    FROM olist_customers_dataset AS c  
    JOIN olist_orders_dataset_new AS o  
        ON c.customer_id = o.customer_id  
    JOIN olist_order_items_dataset AS oi  
        ON o.order_id = oi.order_id  
    GROUP BY  
        c.customer_id,  
        c.customer_state  
)  
state_average AS (  
    SELECT  
        customer_state,  
        AVG(total_spending) AS avg_state_spending  
    FROM customer_spending  
    GROUP BY customer_state  
)  
SELECT  
    cs.customer_id,  
    cs.customer_state,  
    ROUND(cs.total_spending, 2) AS total_spending,  
    ROUND(sa.avg_state_spending, 2) AS state_average  
FROM customer_spending AS cs  
JOIN state_average AS sa  
    ON cs.customer_state = sa.customer_state  
WHERE cs.total_spending > sa.avg_state_spending  
ORDER BY cs.customer_state, cs.total_spending DESC;
```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	customer_id	customer_state	total_spending	state_average				
	b9374bfb8d4215e1fc67d31d2be24191	AM	545.08	188.46				
	b67cda5c14480b9958d2f1465fb3608c	AM	517.20	188.46				
	389efb68674e38efb4ce37a9b72ca36f	AM	488.16	188.46				
	9eec05db89d3d8863da0018d2fac099d	AM	447.09	188.46				
	b482b6eb1e1daa52fc0b17faea2cfb4d	AM	420.13	188.46				
	6b170d60eabcac609caaf419e9e57f1	AM	371.19	188.46				
	2009e285643a9f9aaa39ab0b1a680882	AM	370.29	188.46				
	19dc2be7592f399eb8b754a5b44d847d	AM	334.01	188.46				
	3db8f291923971467c299f44c051a6e9	AM	325.47	188.46				
	4104c2e389807d2511cd9c3874b748f1	AM	313.40	188.46				
	7678332c9841a05031112bc423ee10f1	AM	298.67	188.46				
	bfae4dc0371d8a5a961d6f2658defc5	AM	281.82	188.46				
	...	...	...	...				
Result 16								

22. Get the 2nd highest revenue-generating product in each category.

```
WITH product_revenue AS (  
    SELECT  
        p.product_category_name,  
        oi.product_id,  
        SUM(oi.price + oi.freight_value) AS revenue  
    FROM olist_order_items_dataset AS oi  
    JOIN olist_products_dataset AS p  
        ON oi.product_id = p.product_id  
    GROUP BY  
        p.product_category_name,  
        oi.product_id  
)  
ranked_products AS (  
    SELECT  
        product_category_name,  
        product_id,  
        revenue,  
        DENSE_RANK() OVER (  
            PARTITION BY product_category_name  
            ORDER BY revenue DESC  
        ) AS revenue_rank  
    FROM product_revenue  
)  
SELECT  
    product_category_name,  
    product_id,  
    ROUND(revenue, 2) AS revenue  
FROM ranked_products  
WHERE revenue_rank = 2  
ORDER BY product_category_name;
```

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
product_category_name	product_id	revenue	
	eed5cbd74fac3bd79b7c7ec95fa7507d	10223.82	
agro_industria_e_comercio	423a6644f0aa529e8828ff1f91003690	8609.73	
alimentos	89321f94e35fc6d7903d36f74e351d40	4883.44	
alimentos_bebidas	90f97298579cd20412fdcc9b7a2d4b6b	1605.26	
artes	1bdf5e6731585cf01aa8169c7028d6ad	6726.66	
artes_e_artesanato	6bb18295cc019bf3b6dc7c773411d530	314.32	
artigos_de_festas	833fb04f7b1c82365c0273e62de0c19f	602.58	
artigos_de_natal	f3ccdb9f9c7f31e0efd626e9110b85f5	1379.12	
audio	db5efde3ad0cc579b130d71c4b2db522	10231.39	
automotivo	1dec4c88c685d5a07bf01dcb0f8bf9f8	22228.51	
bebes	cac9e5692471a0700418aa3400b9b2b1	11484.57	
bebidas	0c9439abe84dc40a64fc164365c4d064	3611.44	

23. Create a product category hierarchy using recursive CTEs.

Skipped

24. Find customers purchasing in 3+ consecutive months.

```
WITH customer_months AS (  
    SELECT DISTINCT  
        customer_id,  
        DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS purchase_month  
    FROM olist_orders_dataset_new  
),  
numbered_months AS (  
    SELECT  
        customer_id,  
        purchase_month,  
        LAG(purchase_month, 1) OVER (  
            PARTITION BY customer_id  
            ORDER BY purchase_month  
        ) AS previous_month,  
        LAG(purchase_month, 2) OVER (  
            PARTITION BY customer_id  
            ORDER BY purchase_month  
        ) AS two_months_before  
    FROM customer_months  
)  
SELECT DISTINCT  
    customer_id,  
    purchase_month  
FROM numbered_months  
WHERE TIMESTAMPDIFF(  
    MONTH,  
    STR_TO_DATE(two_months_before, '%Y-%m'),  
    STR_TO_DATE(purchase_month, '%Y-%m')  
    ) = 2  
AND TIMESTAMPDIFF(  
    MONTH,  
    STR_TO_DATE(previous_month, '%Y-%m'),
```

```
STR_TO_DATE(purchase_month, '%Y-%m')
```

```
) = 1
```

```
ORDER BY customer_id, purchase_month;
```

Result Grid		Filter Rows:
customer_id	purchase_month	

Result 23 x

No customers were found who made purchases in three or more consecutive calendar months.

Part F: Advanced Window Functions (4 Questions)

25. Calculate a 7-day moving average of daily order count.

WITH daily_orders AS (

SELECT

DATE(order_purchase_timestamp) AS order_date,

COUNT(DISTINCT order_id) AS order_count

FROM olist_orders_dataset_new

GROUP BY DATE(order_purchase_timestamp)

)

SELECT

order_date,

order_count,

ROUND(

AVG(order_count) OVER (

ORDER BY order_date

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

),

2

) AS moving_avg_7_days

FROM daily_orders

ORDER BY order_date;

Result Grid	Filter Rows:	Export
order_date	order_count	moving_avg_7_days
2016-09-04	1	1.00
2016-09-05	1	1.00
2016-09-13	1	1.00
2016-09-15	1	1.00
2016-10-02	1	1.00
2016-10-03	8	2.17
2016-10-04	63	10.86
2016-10-05	47	17.43
2016-10-06	51	24.57
2016-10-07	46	31.00
2016-10-08	42	36.86
2016-10-09	26	40.43

Result 24 x

26. Find gaps between consecutive customer orders using LAG().

```
WITH customer_orders AS (  
    SELECT  
        customer_id,  
        order_id,  
        order_purchase_timestamp,  
        LAG(order_purchase_timestamp) OVER (  
            PARTITION BY customer_id  
            ORDER BY order_purchase_timestamp  
        ) AS previous_order_date  
    FROM olist_orders_dataset_new  
)  
SELECT  
    customer_id,  
    order_id,  
    order_purchase_timestamp,  
    previous_order_date,  
    DATEDIFF(  
        order_purchase_timestamp,  
        previous_order_date  
    ) AS gap_days  
FROM customer_orders  
WHERE previous_order_date IS NOT NULL  
ORDER BY customer_id, order_purchase_timestamp;
```

No consecutive customer orders were found;
therefore, no order gaps could be calculated.

27. Rank sellers by revenue within each state.

WITH seller_revenue AS (

SELECT

s.seller_id,

s.seller_state,

SUM(oi.price + oi.freight_value) AS revenue

FROM olist_sellers_dataset AS s

JOIN olist_order_items_dataset AS oi

ON s.seller_id = oi.seller_id

GROUP BY

s.seller_id,

s.seller_state

)

SELECT

seller_id,

seller_state,

ROUND(revenue, 2) AS revenue,

RANK() OVER (

PARTITION BY seller_state

ORDER BY revenue DESC

) AS seller_rank

FROM seller_revenue

ORDER BY seller_state, seller_rank;

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	
	seller_id	seller_state	revenue	seller_rank
▶	4be2e7f96b4fd749d52dff41f80e39dd	AC	299.84	1
	327b89b872c14d1c0be7235ef4871685	AM	1258.80	1
	53243585a1d6dc2643021fd1853d8905	BA	235856.68	1
	c72de06d72748d1a0dfb2125be43ba63	BA	18733.18	2
	75d34ebb1bd0bd7dde40dd507b8169c3	BA	16479.51	3
	d03698c2efd04a549382afa6623e27fb	BA	9644.62	4
	4aba391bc3b88717ce08eb11e44937b2	BA	8435.21	5
	a3dd39f583bc80bd8c5901c95878921e	BA	5412.16	6
	1444c08e64d55fb3c25f0f09c07ffcf2	BA	2818.74	7
	659e8466eb3ff1b0e8740d74fb7bbedd	BA	1880.00	8
	d2e753bb80b7d4faa77483ed00edc8ca	BA	1774.94	9
	4fb41dff7c50136976d1a5cf004a42e2	BA	1296.77	10

28. Calculate running revenue totals and percentage contribution.

```
WITH monthly_revenue AS (  
    SELECT  
        DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS month,  
        SUM(oi.price + oi.freight_value) AS revenue  
    FROM olist_orders_dataset_new AS o  
    JOIN olist_order_items_dataset AS oi  
        ON o.order_id = oi.order_id  
    GROUP BY DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')  
)  
SELECT  
    month,  
    ROUND(revenue, 2) AS revenue,  
  
    ROUND(  
        SUM(revenue) OVER (  
            ORDER BY month  
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
        ),  
        2  
    ) AS running_revenue,  
  
    ROUND(  
        (revenue / SUM(revenue) OVER ()) * 100,  
        2  
    ) AS percentage_contribution  
  
FROM monthly_revenue  
ORDER BY month;
```


Part G: Stored Procedures & Optimization (2 Questions)

29. Create a stored procedure for dynamic discount calculation.

```
DELIMITER //
```

```
CREATE PROCEDURE CalculateDiscount(IN order_amount DECIMAL(10,2))
```

```
BEGIN
```

```
    DECLARE discount_rate DECIMAL(5,2);
```

```
    DECLARE discount_amount DECIMAL(10,2);
```

```
    DECLARE final_amount DECIMAL(10,2);
```

```
    IF order_amount >= 2000 THEN
```

```
        SET discount_rate = 15;
```

```
    ELSEIF order_amount >= 1000 THEN
```

```
        SET discount_rate = 10;
```

```
    ELSEIF order_amount >= 500 THEN
```

```
        SET discount_rate = 5;
```

```
    ELSE
```

```
        SET discount_rate = 0;
```

```
    END IF;
```

```
    SET discount_amount = order_amount * discount_rate / 100;
```

```
    SET final_amount = order_amount - discount_amount;
```

```
    SELECT
```

```
        order_amount AS original_amount,
```

```
        discount_rate AS discount_percentage,
```

```
        ROUND(discount_amount, 2) AS discount_amount,
```

```
        ROUND(final_amount, 2) AS final_amount;
```

```
END //
```

```
DELIMITER ;
```

CALL CalculateDiscount(2500);

Result Grid				
Filter Rows:		Export:		
Wrap Cell Content:				
	original_amount	discount_percentage	discount_amount	final_amount
▶	2500.00	15.00	375.00	2125.00

30. Optimize slow queries using EXPLAIN ANALYZE and indexes.

```
SELECT
    o.order_id,
    o.customer_id,
    oi.product_id,
    oi.price
FROM olist_orders_dataset_new AS o
JOIN olist_order_items_dataset AS oi
    ON o.order_id = oi.order_id
WHERE o.customer_id = '00012a2ce6f8dcda20d059ce98491703';
```

```
EXPLAIN ANALYZE
SELECT
    o.order_id,
    o.customer_id,
    oi.product_id,
    oi.price
FROM olist_orders_dataset_new AS o
JOIN olist_order_items_dataset AS oi
    ON o.order_id = oi.order_id
WHERE o.customer_id = '00012a2ce6f8dcda20d059ce98491703';
```

```
CREATE INDEX idx_orders_customer
ON olist_orders_dataset_new(customer_id);
```

```
CREATE INDEX idx_items_order  
ON olist_order_items_dataset(order_id);
```

EXPLAIN ANALYZE

SELECT

```
o.order_id,  
o.customer_id,  
oi.product_id,  
oi.price
```

FROM olist_orders_dataset_new AS o

JOIN olist_order_items_dataset AS oi

```
ON o.order_id = oi.order_id
```

WHERE o.customer_id = '00012a2ce6f8dcda20d059ce98491703';

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
38	19:14:44	EXPLAIN ANALYZE SELECT o.order_id, o.customer_id, oi.product_id, oi.price FROM olist_orders_dataset...	1 row(s) returned	0.078 sec / 0.000 sec
39	19:27:04	CREATE INDEX idx_orders_customer ON olist_orders_dataset_new(customer_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.282 sec
40	19:27:08	CREATE INDEX idx_items_order ON olist_order_items_dataset(order_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.391 sec
41	19:27:27	EXPLAIN ANALYZE SELECT o.order_id, o.customer_id, oi.product_id, oi.price FROM olist_orders_dataset...	1 row(s) returned	0.000 sec / 0.000 sec